

I created an extract, transform, load pipeline.

Main stack: python, pandas, postgres, docker

Pipeline reads files from data/raw folder, exactly 4 with names given in the README cleans them, create analytical table with all columns from every file and loads in the database. Everything is dockerized, so the end user only needs a docker desktop.

how to use:

1. install docker
2. put raw data in data/raw/ folder
3. get in the lesson_13_DE dir in the terminal
4. execute: “docker compose up --build”
5. magic happens
6. now you have 2 docker containers, 1 will deactivate as soon as it loads data into the db, the second is the db itself

Next step would be to create an api, so we can actually load new data to existing db, but that wasn't in the task requirements, so i will limit myself with just loading data into db once.

Detailed description what cleaning and why was done to the data:

1. **customers.csv**: drop the invalid or duplicate email, i assumed we cant have 2 customers from 1 email, drop invalid or duplicate id's, drop invalid date format, and lowercase the emails so Abc and abc email would count as duplicates

2. **orders.csv**: drop rows with invalid date, customer id (if we cant find id in clean customer dataframe it counts as invalid), invalid/duplicated order id, and lowercase the order status
3. **products.csv**: drop rows with invalid/duplicated product ids, name, without category or invalid price
4. **order_item.csv**: Drop rows with invalid\duplicated order\product\order_item id's or where quantity is invalid

I mostly dropped the rows I didn't like, because fixing them wouldn't be possible in most cases and that is just the simplest way to get rid of them.

Output table looks like this

```
<class 'pandas.DataFrame'>
RangeIndex: 906 entries, 0 to 905
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   order_item_id                        906 non-null    int64
1   order_id                            906 non-null    int64
2   product_id                          906 non-null    int64
3   quantity                            906 non-null    int64
4   customer_id                         906 non-null    int32
5   order_status                        906 non-null    str
6   created_at                          906 non-null    datetime64[us]
7   name                                906 non-null    str
8   category                            906 non-null    str
9   price                               906 non-null    float64
10  total_cost                          906 non-null    float64
11  email                               906 non-null    str
12  country                             906 non-null    str
13  customer_registration_date          906 non-null    datetime64[us]
dtypes: datetime64[us](2), float64(2), int32(1), int64(4), str(5)
memory usage: 95.7 KB
```

i merged on id's all given tables into one bigger, that contains all columns, and added 1 more column, the total_cost for every order_item, just quantity*product_price

Code in the directory I believe is easy to install and use, you don't need to resolve anything, all work is done in the docker container by docker.

What results i get from running the docker compose:

```
PS C:\Users\vital\Desktop\DSCamp\lesson_13_DE> docker compose up --build
[+] Building 1.8s (12/12) FINISHED                                docker:desktop-linux
=> [etl internal] load build definition from Dockerfile
=> => transferring dockerfile: 228B                                0.0s
=> [etl internal] load metadata for docker.io/library/python:3.12-slim 1.0s
=> [etl internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                       0.0s
=> [etl 1/6] FROM docker.io/library/python:3.12-slim@sha256:401f6e1a67dad31a1bd78e9ad22d0ee0a3b52154e6bd30e90be696bb 0.1s
=> => resolve docker.io/library/python:3.12-slim@sha256:401f6e1a67dad31a1bd78e9ad22d0ee0a3b52154e6bd30e90be696bb6a3d 0.1s
=> [etl internal] load build context
=> => transferring context: 3.07kB                                    0.0s
=> CACHED [etl 2/6] WORKDIR /lesson_13_DE/app                    0.0s
=> CACHED [etl 3/6] COPY requirements.txt .                       0.0s
=> CACHED [etl 4/6] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [etl 5/6] COPY main.py .                                0.0s
=> [etl 6/6] COPY utils.py .                                      0.1s
=> [etl] exporting to image                                       0.4s
=> => exporting layers                                              0.1s
=> => exporting manifest sha256:cd793a188555bcbdc56b5d6868c1ac911904c1b616117fb02773c857a3061cde 0.0s
=> => exporting config sha256:090c442905c48a5d33ce06c9e167453741e8749e096f54b748c4c063b3b16e64 0.0s
=> => exporting attestation manifest sha256:c22fd1d4e8e30b5979c36d41c2052e1db0eba6db190304aa693f75cdf7877c07 0.1s
=> => exporting manifest list sha256:c0c1f0d4af7e4eaa9532c99b8d429d2379e7445ecafcdcecc8754890db5cc213a 0.0s
=> => naming to docker.io/library/lesson_13_de-etl:latest         0.0s
=> => unpacking to docker.io/library/lesson_13_de-etl:latest     0.0s
=> [etl] resolving provenance for metadata file                   0.0s
```

```
[+] Running 3/3
✔ etl Built 0.0s
✔ Container lesson_13_de-etl-1 Recreated 0.6s
✔ Container postgres_db Recreated 0.3s
Attaching to etl-1, postgres_db
postgres_db |
postgres_db | PostgreSQL Database directory appears to contain a database; Skipping initialization
postgres_db |
postgres_db | 2026-05-14 07:12:54.319 UTC [1] LOG: starting PostgreSQL 17.9 (Debian 17.9-1.pgdg13+1) on x86_64-pc-linux-gn
u, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
postgres_db | 2026-05-14 07:12:54.320 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
postgres_db | 2026-05-14 07:12:54.320 UTC [1] LOG: listening on IPv6 address "::", port 5432
postgres_db | 2026-05-14 07:12:54.335 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
postgres_db | 2026-05-14 07:12:54.353 UTC [29] LOG: database system was shut down at 2026-05-14 07:12:46 UTC
postgres_db | 2026-05-14 07:12:54.368 UTC [1] LOG: database system is ready to accept connections
etl-1 |
etl-1 | order_item_id order_id ... country created_at_y
etl-1 | 0 1 5001 ... IT 2024-01-25 22:15:00
etl-1 | 1 2 5001 ... IT 2024-01-25 22:15:00
etl-1 | 2 3 5002 ... US 2024-03-27 20:41:00
etl-1 | 3 4 5002 ... US 2024-03-27 20:41:00
etl-1 | 4 5 5003 ... NL 2024-01-28 11:51:00
etl-1 | .. ... ... ..
etl-1 | 901 902 5448 ... US 2024-03-17 10:31:00
etl-1 | 902 903 5449 ... CZ 2024-02-07 20:39:00
etl-1 | 903 904 5449 ... CZ 2024-02-07 20:39:00
etl-1 | 904 905 5449 ... CZ 2024-02-07 20:39:00
etl-1 | 905 906 5450 ... SE 2024-03-30 08:35:00
etl-1 |
etl-1 | [906 rows x 13 columns]
etl-1 exited with code 0
```

the etl-1 container loads the data in DB, then retrieves it and prints it out in the console. So if we see the table it means everything works as planned.

Dockerfile basically copies the app folder, gets all requirements and runs it, the compose is telling what containers to create and how to run them, also loading data into volume, instead of copying it into the container itself.